A digital vault is locked with an unknown 10-bit binary combination. The vault has a glitch: whenever you input a sequence, it returns a score indicating how **close** you are to the correct key.

Your goal is to use a Genetic Algorithm to try and find the Master Key: **a string consisting entirely of 1s.**

1. Complete the *create_population* function. There will be 4 chromosomes, each having a randomly generated 10 bit number consisting only 0 and 1. (Example- a gene can look like this '0100110111'

2. Find the *fitness function* which will tell you how close each chromosome is to the master key.

3. Write the code for *selection* function. Use random selection.

```python
import random

CHROMOSOME_LENGTH = 10
POPULATION_SIZE = 4
MUTATION_RATE = 0.1
MAX_GENERATIONS = 3000      # stop if not found by then
MASTER_KEY = "1" * CHROMOSOME_LENGTH   # 1111111111


# ---------------- Step 1: Create the population ----------------
def create_population(size=POPULATION_SIZE, length=CHROMOSOME_LENGTH):
    """Return a list of `size` random chromosomes.
    Each chromosome is a string of `length` random 0s and 1s, e.g. '0100110111'."""
    population = []
    # TODO

    return population


# ---------------- Step 2: Fitness function ----------------
def fitness(chromosome):
    """Number of positions that match the master key."""
    fitness_score = 0
    # TODO

    return fitness_score


# ---------------- Selection ----------------
def selection(population):
    """Random selection: pick one chromosome from the population at random,
    without looking at fitness."""
    # TODO
    # (The line below will give an error until you write your code above.)

    return chromosome   # you may change this variable


# ---------------- Step 3a: Single-point crossover ----------------
def crossover(parent1, parent2):
    """Cut both parents at the same random point and swap the tails."""
    point = random.randint(1, len(parent1) - 1)
    child1 = parent1[:point] + parent2[point:]
    child2 = parent2[:point] + parent1[point:]
    return child1, child2
```

```python
# ---------------- Step 3b: Mutation ----------------
def mutation(chromosome, rate=MUTATION_RATE):
    """Each bit has a `rate` chance of being flipped."""
    new_chromosome = ""
    for bit in chromosome:
        if random.random() < rate:
            new_chromosome += "1" if bit == "0" else "0"   # flip
        else:
            new_chromosome += bit
    return new_chromosome


# ---------------- Main GA loop (do not change) ----------------
def genetic_algorithm():
    population = create_population()

    for generation in range(MAX_GENERATIONS):
        scores = [fitness(c) for c in population]
        print(f"Gen {generation:3d}: {population}  fitness = {scores}")

        # Check if the master key is in the current population
        if MASTER_KEY in population:
            print(f"\nVault unlocked! Master Key {MASTER_KEY} "
                  f"found in generation {generation}.")
            return MASTER_KEY, generation

        # 2 parents -> 2 children, done twice, so the population stays at 4
        new_population = []
        while len(new_population) < POPULATION_SIZE:
            parent1 = selection(population)
            parent2 = selection(population)
            child1, child2 = crossover(parent1, parent2)
            new_population.append(mutation(child1))
            new_population.append(mutation(child2))

        population = new_population[:POPULATION_SIZE]  # keep exactly 4

    best = max(population, key=fitness)
    print(f"\nNot found in {MAX_GENERATIONS} generations. "
          f"Best: {best} (fitness {fitness(best)})")
    return best, MAX_GENERATIONS


if __name__ == "__main__":
    genetic_algorithm()
```